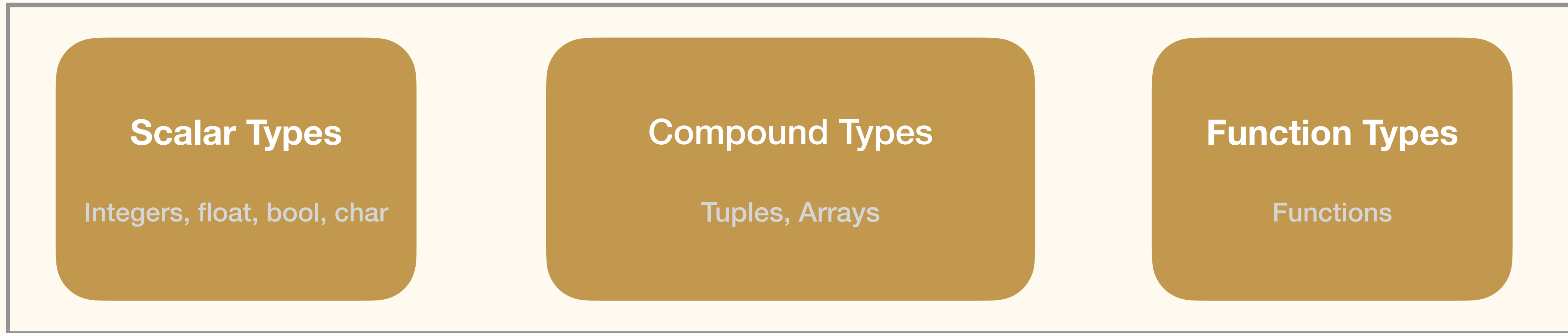


VARIABLES, MUTABILITY & FUNCTIONS

Definition

Rust uses **data types** to define what **kind of values** a variable can store. As a **statically typed** language, it checks types at **compile time**, often requiring explicit annotations.

7 types of Data types



1. Scalar types

Category	Types
Integer	<code>i8</code> , <code>u8</code> , <code>i16</code> , <code>u16</code> , <code>i32</code> , <code>u32</code> , <code>i64</code> , <code>u64</code> , <code>i128</code> , <code>u128</code> , <code>isize</code> , <code>usize</code>
Floating	<code>f32</code> , <code>f64</code>
Character	<code>char</code>
Boolean	<code>bool</code>

Length	Signed	Unsigned
8-bit	<code>i8</code>	<code>u8</code>
16-bit	<code>i16</code>	<code>u16</code>
32-bit	<code>i32</code>	<code>u32</code>
64-bit	<code>i64</code>	<code>u64</code>
128-bit	<code>i128</code>	<code>u128</code>
arch	<code>isize</code>	<code>usize</code>

DATA TYPE	MIN	MAX	Length
i8	-128	127	8-bit
i16	-32768	32767	16-bit
i32	-2147483648	2147483647	32-bit
i64	-92233720368547...	92233720368547...	64-bit
i128	-17014118346046923173168...	17014118346046923173168...	128-bit

2. Compound types

Category	Types
Tuple	<code>(i32, f64, char)</code>
Array	<code>[i32; 3]</code> , <code>[bool; 5]</code> , etc.

3. Function types

Category	Types
Function Pointer	<code>fn(i32, i32) -> i32</code>
Closure	<code>let f = x x + 1;</code>

Exercise 1: Print variables



```
let name = "Rustacean";
```



```
let age: u8 = 25;
```



```
println!("Name: {name}, Age: {age}, Language: {language}, Year: {year}");
```



```
println!("I love {0} and {1}, but I prefer {0}", rust, javascript);
```


Exercise 2: Variable shadowing

Variable shadowing



```
let number = 5;  
let number = number + 1; // shadowing
```

Exercise 3: Mut variables

By default, each variable is **immutable** in Rust.

Mutability



```
let mut level = 5;  
// need to make it mutable  
level = level + 1;
```


Exercise 4: Let vs const

Exercise 5: Scopes & Blocks

Definition

A **block** is the **area** between an **opening curly brace** and a **closing curly brace**.

Scope (inner block)

```
● ● ●  
  
{  
    // Start of inner block (scope)  
    let y = 20;  
    let x = 99; // shadows outer x  
    println!("Inside block: x = {}, y = {}", x, y);  
} // End of inner block – y is dropped here
```

✗ Scope errors, example 1



```
fn main() {  
    {  
        let x = 42; // x is only valid inside this block  
    }  
  
    println!("x is: {}", x); // ✗ Error: x is out of scope here  
}
```

✗ Scope errors, example 2

```
fn main() {  
    let x = 10; // outer x  
  
    {  
        let y = x + 5;  
        println!("Inside block: y = {}", y);  
    }  
  
    println!("Outside block: y = {}", y); // ✗ ERROR: y is not in scope  
}
```


✗ Scope errors, example 3

```
fn main() {  
    let x = 5;  
  
    {  
        let y = 10;  
        println!("Inside block: x = {}, y = {}", x, y);  
    }  
  
    // Scope error! y is not available here:  
    println!("Outside block: x = {}, y = {}", x, y);  
}
```

Exercise 6: Functions in Rust

Function in Rust



```
fn say_hello() {  
    println!("Hello, Rust!");  
}
```

Function with one argument



```
fn say_hello(name: &str) {  
    println!("Hello, {}!", name);  
}
```

Function that returns a String



```
fn greet(name: &str) -> String {  
    format!("Hello, {}!", name)  
}
```


Exercise 7: Arrays

Arrays



```
let names = ["Alice", "Bob", "Charlie"];
```

Arrays (typed)



```
let names: [&str; 3] = ["Alice", "Bob", "Charlie"];
```


Exercise 8: Tuples

Tuples



```
let person = ("Alice", 30);
```


Tuples (typed)



```
let person: (&str, i32) = ("Alice", 30);
```

Destructuring data from a tuple



```
fn main() {  
    let product: (&str, f64, u32) = ("Laptop", 999.99, 5);  
  
    // Destructure the tuple  
    let (name, price, stock) = product;  
  
    println!("Product: {name}, price: ${price}, in stock: {stock}");  
}
```

A Quick recap

7 types of Data types

Scalar Types

Integers, float, bool, char

Compound Types

Tuples, Arrays

Function Types

Functions

Collection Types

Vec, String, Hashmap, ...

Custom Types

Enum, struct

Special Types

Unit types, &T, &mut T

Smart pointers

Box, Rc, Arc, RefCell, ...

Your turn!